

Day 1 — Foundations: From State to Meaning

Curriculum alignment: Stage 1 — Foundations of Computation · Electricity, States, Bits, and Information

Target time: approximately 3 hours

- Part I — physical state and reliable bits: about 50 minutes
- Part II — patterns, notation, and meaning: about 55 minutes
- Part III — Linux observation and integrated challenge: about 55 minutes
- Mastery explanation and review: about 20 minutes

A computer can display a photograph, execute an instruction, add two numbers, and print a sentence. Yet at its foundation it has only physical things changing state. How can such a machine represent anything at all?

That is the problem for Day 1.

We will not begin by declaring that computers “speak binary.” That phrase hides every interesting engineering decision. We will begin lower down, with a real physical machine in a noisy world, and build the chain ourselves:

```
physical state
  → reliable logical alternatives
  → bit patterns
  → encodings
  → useful meaning
```

By the end, you should be able to explain why the same stored pattern can be a number, a character, an instruction, or a row of pixels—and why that fact is the necessary starting point for logic circuits and CPUs.

1. The old problem beneath the modern machine

Before electronics, calculation was work done by people. Fingers, tally marks, counting boards, and abaci helped because they moved part of the work out of a person's memory and into the state of a physical object. A bead on one side of a rod could preserve a partial result while the person considered the next step.

Mechanical calculators pushed the idea further. Wheels and gears could be arranged so that one complete rotation carried into the next position. The machine's geometry enforced part of the arithmetic rule. Later designs made storage, arithmetic, and control increasingly distinct mechanisms.

The ambition was already recognizable: preserve facts, transform them by repeatable rules, and avoid asking a human to perform every intermediate step.

But the medium imposed limits. Precision mechanisms were expensive, slow, subject to wear, and difficult to scale. Adding more interacting states meant adding more mechanical complexity.

Punched media solved a different part of the problem. A loom could use holes in cards to select weaving operations. Telegraph codes could turn characters into patterns that traveled through a narrow communication channel. Hollerith's census machinery encoded facts by the positions of holes in cards. When a reader pin passed through a hole, it completed an electrical path; where there was no hole, it did not.

Notice the causal progression:

```
human memory is limited
  → put state into a durable physical object

manual transformation is slow
  → build mechanisms whose structure enforces a rule

facts are too varied for one fixed mechanism
  → encode them as reusable patterns

mechanical motion is slow and difficult to scale
  → use electrical switching
```

No single person or machine suddenly invented the modern computer. Many developments solved different constraints: representation, switching, storage, arithmetic, control, and programmability. Today we establish the representation layer on which the rest depends.

2. Computation starts with physical state

A useful first model of computation is:

```
current physical state + input + rule → new physical state
```

An abacus changes bead positions. A mechanical calculator changes gear positions. A relay machine changes which contacts are open or closed. A modern processor changes charge and voltage across networks of transistors.

The materials differ, but computation always needs real state. A value that can affect later events cannot be stored “nowhere.” It must be represented by some physical condition:

- the position of an object,
- an open or closed electrical contact,
- voltage on a conductor,
- charge in a capacitor,
- magnetic orientation,
- or another distinguishable physical state.

Software is not exempt from physics. A variable in C is an abstraction, but while the program runs, its value is represented in registers, caches, or memory by physical states. A file is an abstraction, but its data ultimately depends on states in storage devices.

What exactly is a state?

A **state** is the condition of a system at a particular moment.

Consider a simple switch:

physical position:	DOWN	UP
assigned symbol:	0	1

The positions are physical. The symbols are our chosen labels. Nature does not insist that DOWN means zero; we could reverse the mapping and the switch would behave exactly as before.

This distinction will remain important throughout systems engineering:

what physically exists $\neq$ what our convention says it means

For a state to be useful in a computer, it should be:

1. distinguishable from alternatives,
2. stable long enough to be used,
3. intentionally changeable,
4. readable with acceptably low ambiguity.

Those demands—not a mystical preference for zero and one—lead us toward digital electronics.

3. Electricity is continuous; digital logic is engineered

A common shortcut says:

0 volts = 0
3.3 volts = 1

That is a useful cartoon, but it is not how a real input specification works. Voltage is continuous. A wire may be at 0.00 V, 0.19 V, 1.42 V, 2.87 V, or any of countless values between. Real circuits also face:

- electrical noise,
- coupling from nearby signals,
- resistance and voltage drop,
- power-supply variation,
- manufacturing differences,
- and changing temperature.

A practical digital receiver therefore does not demand two perfect voltage points. It recognizes ranges. As an illustrative example:

0.0 V	—————	0.8 V		2.0 V	—————	3.3 V
valid LOW				valid HIGH		
→ 0		unspecified		→ 1		
		region				

The actual thresholds depend on the device and logic family. The durable idea is this:

Logical zero and one are produced by classifying broad ranges of imperfect physical values.

Suppose a sender intends a LOW signal near 0.2 V and noise adds 0.3 V. The receiver sees 0.5 V, which is still safely in the LOW range. The separation between what a sender guarantees and what a receiver will accept provides a **noise margin**.

What about 1.4 V in the middle region? The specification does not promise a reliable logical result there. One device might read LOW while another reads HIGH; the answer may also vary with temperature or timing. Circuit designers arrange valid signals to pass through that region quickly and settle into a valid range.

The clean bit is therefore an abstraction boundary:

continuous, imperfect physical quantity
↓ thresholding and restoration
stable logical alternative: 0 or 1

Above that boundary, we can reason about exact logical values. Below it, engineers still have to manage analog behavior.

Why only two logical states?

Imagine dividing a 0-to-3.3-volt range into ten bands so one wire could directly represent a decimal digit. Adjacent values would be close together. Small disturbances could push a signal into the next band, and the receiver would need to distinguish many levels accurately.

Two states use the available physical range more generously:

many levels	two levels
-----	-----
more information per element	less information per element
narrower separation	wider noise margin
more precise receiver	simpler switching decision
harder restoration	easier restoration

Binary is not the only possible choice. Multi-level storage exists and can increase density, but it pays with tighter margins, more complex sensing, slower operations, wear management, or error correction.

Binary became foundational because it is an excellent engineering bargain:

Give up density in one physical element to gain reliable discrimination, simple switching, and easy regeneration. Recover capacity by combining many elements.

4. From one bit to many patterns

A **bit** is one binary digit: one resolved choice between two logical alternatives.

The bit is not the voltage itself. Voltage is one possible implementation:

PHYSICAL IMPLEMENTATION		LOGICAL VALUE
low/high voltage	→	0 / 1
uncharged/charged storage cell	→	0 / 1
magnetic orientation A/B	→	0 / 1
open/closed relay	→	0 / 1

One bit has two possible patterns:

0
1

With two independent bits, each old pattern has two extensions:

00 01 10 11

With three:

000 001 010 011 100 101 110 111

Each added independent bit doubles the possibilities. Every existing pattern gets one version with the new bit equal to zero and one with it equal to one:

n bits → 2^n distinct patterns

Do not treat the exponent as magic notation. It records repeated two-way choice:

$2 \times 2 \times 2 \times \dots \times 2 = 2^n$

This also solves the reverse problem. To distinguish at least 20 states:

4 bits → 16 patterns (not enough)
5 bits → 32 patterns (enough)

An encoding may leave some of those 32 patterns unused or reserve them for special purposes.

Bits, bytes, and words

A **byte** is eight bits grouped together:

01000110

Eight bits provide $2^8 = 256$ patterns. The eight-bit byte is a successful historical convention, not a law of physics.

A **word** is a context-dependent natural data width for an architecture. A modern processor may have 64-bit general-purpose registers, while an instruction-set manual may retain older names such as *word* for 16 bits, *doubleword* for 32 bits, and *quadword* for 64 bits. Ask which architecture and which convention are in use.

The hierarchy so far is:

physical state → logical bit → grouped bit pattern

But a pattern is not yet a number, a letter, or an instruction.

5. Positional notation: decimal, binary, and hexadecimal

We already use positional encoding every day. In decimal:

$$347 = 3 \times 10^2 + 4 \times 10^1 + 7 \times 10^0$$

A digit's contribution depends on both the digit and its position. Binary uses the same idea with base two:

bit position:	3	2	1	0
place value:	8	4	2	1
pattern:	1	0	1	1
value:	$8 + 0 + 2 + 1 = 11$			

Under the **unsigned binary integer encoding**, 1011_2 means decimal 11. The pattern does not intrinsically contain “eleven.” We selected a rule that assigns powers-of-two place values.

Binary to decimal

Write the powers of two and add the active positions:

10110110_2

bits:	1	0	1	1	0	1	1	0
place values:	128	64	32	16	8	4	2	1

$128 + 32 + 16 + 4 + 2 = 182$

Decimal to binary

Find powers of two that sum to the value:

$$\begin{aligned} 70 &= 64 + 4 + 2 \\ &= 01000110_2 \end{aligned}$$

You can also repeatedly divide by two and collect remainders, but the powers-of-two method makes the representation easier to understand.

Why hexadecimal is useful

Long binary patterns are difficult for humans to scan. Decimal is compact, but it hides bit boundaries. Hexadecimal works especially well because:

$$16 = 2^4$$

Each hexadecimal digit corresponds exactly to four bits:

```
binary:      1111 1010 1101 1110
hexadecimal: F    A    D    E
```

To convert binary to hex, group bits from the right in sets of four:

```
101101102 = 1011 01102 = 0xB6
```

This is why addresses, register values, masks, memory dumps, and instruction bytes are commonly shown in hex. Hexadecimal is not a different physical layer. It is a compact human notation for the same bit pattern.

Keep these statements separate:

```
010001102    binary notation
70             decimal notation
0x46           hexadecimal notation
```

Under an unsigned-integer rule, all three written forms denote the same numeric value.

6. Pattern first, meaning second

Now return to the byte:

```
01000110
```

Hand it to several readers:

```
01000110 ————┐ unsigned integer rule → 70
                  └ ASCII rule → F
                  └ bitmap rule → .#...##.
                  └ instruction decoder → part of an operation
```

The bits did not change. The interpretation did.

An **encoding** is an agreed mapping between patterns and meanings:

```
pattern + interpretation rule → meaning
```

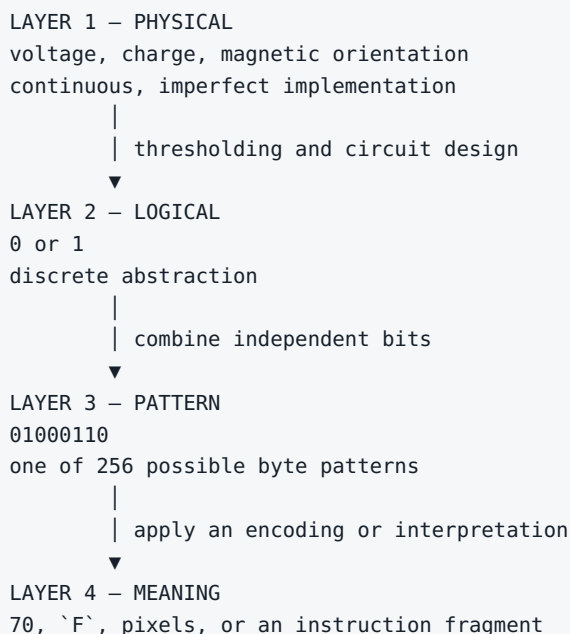
That idea did not begin with electronic computers. Writing maps marks to language. Braille maps raised-dot patterns to symbols. Telegraph systems map signal patterns to characters. Punched-card systems map hole positions to facts or operations.

Hollerith's tabulating system makes the bridge particularly clear:

```
fact about a person
→ clerk follows a card encoding
→ holes occupy selected positions
→ reader senses hole/no-hole electrically
→ machinery counts or sorts the encoded fact
```

A hole did not inherently mean an occupation or an age category. Its meaning came from the card format and the machinery built to interpret that format. A modern file format, character encoding, or instruction set is vastly more capable, but it relies on the same separation between carrier and convention.

The four-layer model



Hardware can be the interpreter. A CPU's instruction decoder applies the instruction set's encoding. Software can also be the interpreter. A terminal applies character conventions; an image viewer applies a file and pixel format.

The pattern itself does not announce which reader should be used.

7. Information is not the same as storage

We often say that a device “stores information,” but storage capacity and information are not identical.

Imagine one physical bit permanently wired to zero. It occupies a bit-sized place in a design, but reading it tells you nothing you did not already know. There was no uncertainty about its value.

Now imagine a fair coin toss stored in one bit. Before reading it, either outcome is possible. After reading, one two-way uncertainty has been resolved.

At this level, think of **information** as a reduction in uncertainty. Think of **storage** as the physical capacity to preserve distinguishable patterns.

That distinction prevents two common mistakes:

1. A large storage area need not contain much new information. A million repeated zeros are highly predictable.
2. Meaning is not measured merely by counting storage cells. The same byte can participate in very different messages under different encodings.

Later, compression and information theory will make this quantitative. For now, preserve the conceptual boundary:

```
storage provides possible distinguishable states
information depends on which uncertainty those states resolve
meaning depends on an interpretation and context
```

8. Observe one pattern through several Linux readers

Now let the machine expose the abstraction layers. We will create four bytes once and then refuse to change them:

```
printf 'FADE' > /tmp/day1-pattern.bin
xxd -b /tmp/day1-pattern.bin
```

The bytes are:

```
01000110 01000001 01000100 01000101
```

Before each command, predict what kind of interpretation it requests.

View the bytes as numbers, hex, and characters

```
od -An -tu1 -tx1 -tc /tmp/day1-pattern.bin
```

The views should correspond to:

70 65 68 69	unsigned decimal byte values
46 41 44 45	hexadecimal byte values
F A D E	characters

The file has not changed between output rows.

View the bytes as instructions

```
objdump -D -b binary -m i386:x86-64 /tmp/day1-pattern.bin
```

The decoder applies the x86-64 instruction encoding. These bytes were not chosen to form a useful program, so do not memorize the resulting assembly. Observe the more important fact: the tool treats the same bytes as machine code rather than text.

View each byte as eight pixels

```
python3 - <<'PY'
for byte in open('/tmp/day1-pattern.bin', 'rb').read():
    row = ' '.join('#' if (byte >> bit) & 1 else '.'
                for bit in range(7, -1, -1))
    print(row)
PY
```

Expected rows:

```
. # . . . # # .  
. # . . . . . #  
. # . . . # . .  
. # . . . # . #
```

The Python code supplied a bitmap convention: one means `#`, zero means `.`, and the most significant bit is displayed first.

The one-pattern-many-meanings observation

This small experiment reaches surprisingly far into Linux:

- A debugger can display the same memory as hex, signed or unsigned numbers, characters, addresses, or instructions.
- A type tells compiled software how to interpret bits and which operations make sense; the type is not painted onto the RAM cells.
- A filename extension does not transform bytes. It helps select a program that knows an expected format.
- A memory-safety exploit may try to make bytes supplied as data become bytes fetched as instructions. The electrical storage still works; the dangerous change is interpretation and control flow.

When a low-level value looks wrong, ask:

1. What bits are actually present?
2. Which reader and encoding are interpreting them?

That habit is useful in debuggers, kernel logs, packet traces, memory dumps, and disassemblers.

9. Integrated challenge — rebuild the whole chain

Use paper first. Predict before running commands. Do not check earlier examples until you have committed to an answer.

Part A — physical reality

Assume a device guarantees:

```
LOW:  at or below 0.8 V  
HIGH: at or above 2.0 V
```

1. Classify 0.3 V, 2.8 V, and 1.4 V.
2. Explain why the middle value must not be trusted.
3. A LOW signal is sent at 0.2 V and gains 0.4 V of noise. What logical value should the receiver still recognize?
4. Explain why ranges make a large digital system more reliable than exact voltage points would.
5. Give one reason a designer might still choose multiple physical levels per cell.

Part B — patterns and capacity

1. List all three-bit patterns.
2. Explain why there are eight without merely quoting 2^3 .
3. How many patterns do six bits provide?
4. What is the minimum number of bits needed to distinguish 50 states?
5. If an encoding uses only 40 of those available patterns, what could the remaining patterns be used for?

Part C — conversions

Convert without a calculator and show your place values or nibble groups:

1. 00101101_2 to decimal and hexadecimal.
2. $0xA7$ to binary and decimal.
3. decimal 99 to binary and hexadecimal.
4. 10110110_2 to decimal and hexadecimal.
5. Explain why hexadecimal preserves visible bit groupings better than decimal.

For every mistake, identify the cause: wrong place value, arithmetic slip, incorrect grouping, or confusion between a pattern and its interpretation.

Part D — predict, observe, explain

Create one byte:

```
printf '\x41' > /tmp/day1-check.bin
```

Write your predictions before running:

```
xxd -b /tmp/day1-check.bin
od -An -tu1 /tmp/day1-check.bin
cat /tmp/day1-check.bin
objdump -D -b binary -m i386:x86-64 /tmp/day1-check.bin
```

Then answer:

1. Did any command alter the file?
2. Which command requested binary notation?
3. Which requested an unsigned numeric interpretation?
4. Which caused the terminal to present a character?
5. Why did the instruction decoder not treat the byte as text?
6. Which result surprised you, and which layer had you accidentally assumed?

Choose a second byte yourself. Correctly predict its binary, decimal, hex, character, and instruction-decoder views before testing it.

Part E — repair this explanation

Rewrite the following so every noun belongs to the correct layer:

A computer stores zero as exactly 0 volts and one as exactly 3.3 volts. Eight voltages make a byte. The byte 01000110 is the letter F.

Your repaired version must mention:

- physical voltage ranges,
- logical bits,
- a byte pattern,
- and the interpretation supplied by ASCII.

10. Mastery checkpoint

Close the lesson and take a blank sheet.

Explain

Answer from first principles:

1. Why is electricity not naturally binary?
2. Why are two broad logical ranges useful in a noisy physical system?
3. Why does each additional independent bit double the number of patterns?
4. What is the difference between a bit pattern and an encoding?
5. Why can one pattern represent a number, character, instruction, or pixel?
6. What is the difference between storage capacity and information?

Draw

Reconstruct and annotate every arrow:

physical state → logical bit → bit pattern → interpretation → meaning

Add one concrete example beneath each stage.

Observe

Reproduce the Linux experiment with a byte of your choosing. Show the same stored byte in binary, decimal, hexadecimal, text, and instruction-decoder views. Explain why the outputs differ.

Build

Invent an encoding for eight possible machine states:

- assign each state a three-bit pattern,
- reserve at least one pattern for an error or future use,
- describe the physical states that could implement the bits,
- and explain what reader would turn the patterns into meaning.

The central answer

You have mastered the foundation when you can express this idea in your own words:

A physical device preserves distinguishable states. Circuit design maps ranges of imperfect physical values into logical bits. Groups of bits form patterns, but storage does not attach semantic labels to them. An encoding, together with a software or hardware reader, maps a pattern to a number, character, instruction, pixel, or another meaning.

Do not mark the checkpoint complete because the paragraph looks familiar. Complete it when you can rebuild the causal chain, perform the conversions, and explain your observed Linux output without the notes.

11. Why this foundation matters to a CPU

We can now represent inputs and results, but we have not yet built a machine that computes with them.

Suppose we want hardware to add two encoded numbers. Several new problems appear:

1. For every possible input pattern, which output pattern should be produced?
2. How can switches implement rules such as AND, OR, and NOT?
3. How can those rules be composed into arithmetic?
4. Where can an intermediate result remain while the next operation occurs?
5. What selects addition rather than another operation?
6. What determines the order of operations?

Those questions lead from bits to logic gates, from gates to adders, from adders to a datapath, and from stored control patterns to machine instructions:

```
reliable representation
+ logic
+ arithmetic
+ storage
+ control
+ encoded instructions
= a programmable processor
```

A conventional CPU does not literally “think.” Physical states change according to circuits and encoded instructions. The remarkable behavior of applications, operating systems, and learning systems emerges from enormous compositions of those simple, controlled transitions.

The point of Day 1 is therefore larger than learning binary conversion. We have established how a noisy physical world can support stable symbols and how stable symbols can carry different meanings. Next, we can ask how a circuit transforms those symbols according to a rule.

References used selectively

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 2–3 for codes and combinations, Chapter 9 for bits, and Chapter 15 for bytes and hexadecimal (local PDF chapter openings at pages 18, 24, 80, and 199).

- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd edition, Chapter 2, “Bits, Data Types, and Operations,” book pages 21–36.
- IBM, “The punched card tabulator,” for Hollerith's use of hole/no-hole patterns and electrical contacts to mechanize census processing: <https://www.ibm.com/history/punched-card-tabulator>
- Computer History Museum, “Computers Timeline,” for the broader transition from mechanical calculation through relay and electronic machines: <https://www.computerhistory.org/timeline/computers/>

The references provide historical and technical grounding. The causal narrative, Linux observations, and integrated challenge are synthesized for this masterclass.